Artificial Intelligence

BS (CS) _SPRING_2026

Assignment # 2

Task 1: Knapsack Problem Using Genetic Algorithm

The **Knapsack Problem** is a classic optimization problem where you have a set of items, each with a weight and a value. You are given a knapsack with a weight capacity, and the goal is to select a subset of items that maximizes the total value without exceeding the weight capacity of the knapsack.

In this task, you will use a **Genetic Algorithm (GA)** to solve a **0/1 Knapsack Problem**. Each individual in the population will represent a potential solution, where each gene in the chromosome represents whether a particular item is included (1) or excluded (0) from the knapsack.

You are given a set of n items, each with a weight w[i], a value v[i], and a size s[i], where 1 <= i <= n.

Your task is to select a subset of the items that maximize the total value while staying within the physical volume capacity of the knapsack. The total weight and size of the selected items must be less than or equal to the weight allowed. The weight constraint is **10 Kg**.

Items	N1	N2	N3	N4	N5	N6
Value	14	23	8	9	17	15
Weight	1	3	7	4	5	6

1. **Initialization:** Generate an initial population of pop_size individuals, where each individual is a binary string of length n that represents whether each item is included or not (1 = included, 0 = not included).
2. **Fitness Function:** Evaluate the fitness of each individual by calculating the total value of the selected items and penalizing solutions that exceed the physical volume capacity of the knapsack. The fitness function should return a value that reflects the quality of the solution.
3. **Selection:** Select two individuals from the population with probability proportional to their fitness. You should use roulette wheel selection.
4. **Crossover:** Generate two offspring by applying crossover to the selected parents. You can use any crossover method you like, such as one-point crossover, two-point crossover, or uniform crossover.
5. **Mutation:** Mutate the offspring by flipping each bit in the binary string with probability **mut_prob.**
6. **Replacement:** Replace the least-fit individuals in the population with the offspring.
7. **Termination:** Repeat steps 3-6 for **max_gen** generations, or until a satisfactory solution is found.

Task 2 Background: What is a CSP?

A **Constraint Satisfaction Problem (CSP)** is defined by a triple `(X, D, C)` where:

1. X = {X1, X2, …, Xn} is a set of variables.

2. D = {D1, D2, …, Dn} is the set of domains, one per variable.
3. C = {C1, C2, …, Cm} is a set of constraints, each restricting the values variables can take simultaneously.

The goal is to find an assignment of values to all variables such that every constraint is satisfied. The classic examples you will test your solver against are:

Problem	Variables	Constraints
Map Coloring	WA, NT, SA, Q, NSW, V, T	WA≠NT, WA≠SA, NT≠SA, NT≠Q, SA≠Q, SA≠NSW, SA≠V, Q≠NSW, NSW≠V
N-Queens	Q1…Qn (column of queen in row i)	Qi≠Qj, |Qi−Qj|≠|i−j| for all i≠j
Sudoku	81 cells, domain {1..9}	All-different in each row, column, and 3×3 box

---

A — CSP Class Design [15 Marks]

Design a general-purpose **CSPSolver** class that can represent and solve any binary CSP. The class must support the following interface exactly (do not rename methods):

Required Class Structure

You must implement the following skeleton (fill in the method bodies):

```
class CSPSolver:
    def __init__(self):
        """
        Initialize the CSP.
        self.variables  -> list of variable names (strings)
        self.domains    -> dict: variable -> list of possible values
        self.constraints -> dict: (var1, var2) -> constraint_function
        """
        self.variables   = []
        self.domains     = {}
        self.constraints = {}
        self.backtracks  = 0   # counter — increment on every backtrack

    def add_variable(self, var, domain):
        """Add a variable with its domain."""
        # TODO: implement

    def add_constraint(self, var1, var2, constraint_fn):
        """
```

```
        Add a binary constraint between var1 and var2.
        constraint_fn(val1, val2) -> bool
        """
        # TODO: implement

    def is_consistent(self, var, value, assignment):
        """
        Return True if assigning `value` to `var` does not
        violate any constraint given the current `assignment`.
        """
        # TODO: implement

    def select_unassigned_variable(self, assignment, use_heuristics=False):
        """
        Return the next unassigned variable.
        If use_heuristics=False: return first unassigned variable.
        If use_heuristics=True:  apply MRV then Degree heuristic (Task 4).
        """
        # TODO: implement

    def order_domain_values(self, var, assignment):
        """Return domain values for var (simple: original order)."""
        # TODO: implement

    def backtrack(self, assignment, use_fc=False, use_heuristics=False):
        """
        Core recursive backtracking search.
        use_fc=True  -> apply Forward Checking on each assignment.
        Returns completed assignment dict or None if no solution.
        """
        # TODO: implement

    def solve(self, use_fc=False, use_heuristics=False):
        """
        Entry point. Resets backtrack counter, calls backtrack().
        Prints total backtracks after solving.
        Returns solution dict or None.
        """
        # TODO: implement
```

Marks Breakdown — Task 1

4. Correct class with all method signatures present and callable: 4 marks
5. add_variable and add_constraint storing data correctly: 4 marks
6. is_consistent checking all relevant constraints: 4 marks
7. Docstrings present on all methods, code is commented: 3 marks

B — Backtracking Search with Forward Checking [20 Marks]

Implement the **backtrack()** method with two modes: (1) plain backtracking and (2) backtracking with Forward Checking.

Part A — Plain Backtracking [10 Marks]

Implement standard backtracking search:

- If assignment is complete (all variables assigned), return it.
- Select the next unassigned variable using select_unassigned_variable().
- For each value in the variable's domain, check consistency using is_consistent().
- If consistent, assign the value and recurse.
- If recursion fails, unassign (backtrack) and increment self.backtracks.
- If no value works, return None.

Part B — Forward Checking [10 Marks]

When **use_fc=True**, after assigning a value to a variable, apply Forward Checking:

- For every unassigned neighbor variable Xj that shares a constraint with Xi:
- Remove from Xj's domain any value that is inconsistent with Xi's assigned value.
- If Xj's domain becomes empty, immediately backtrack (restore pruned values first).
- If all neighbors still have non-empty domains, continue the search.

Important: *You must restore pruned domain values when backtracking. Use a deep copy or an explicit restore mechanism.*

Required Output Format

Your solve() method must print the following for each run:

```
Solving: Australia Map Coloring
Mode: Backtracking + Forward Checking + MRV
Solution: {'WA': 'R', 'NT': 'G', 'SA': 'B', 'Q': 'R', 'NSW': 'G', 'V': 'R',
'T': 'G'}
Backtracks: 3
Time: 0.0012s
```

Marks Breakdown — Task 2

8. Correct recursive backtracking returning valid solution: 5 marks
9. Backtrack counter correctly incremented and printed: 2 marks
10. Forward Checking correctly prunes domains: 6 marks

11. Domain restoration on backtrack (no stale pruning): 4 marks
12. Output format matches specification: 3 marks

C — AC-3 Arc Consistency [15 Marks]

Implement AC-3 as a standalone method **ac3(self)** that enforces arc consistency on the CSP before search begins.

Algorithm Specification

- Initialize a queue with ALL arcs (Xi, Xj) for every constraint.
- While the queue is not empty:
- Dequeue arc (Xi, Xj).
- Call REVISE(Xi, Xj): remove every value v from Di such that there is NO value w in Dj with constraint(v, w) == True.
- If Di was revised (any value removed):
- If Di is now empty: return False (CSP is unsolvable).
- For every Xk that has a constraint with Xi (k ≠ j): enqueue (Xk, Xi).
- Return True (all arcs are arc-consistent).

Required Method Signatures

```
    def revise(self, xi, xj):
        """
        Remove values from self.domains[xi] that have no
        support in self.domains[xj].
        Returns True if domain of xi was revised, False otherwise.
        """
        # TODO: implement


    def ac3(self):
        """
        Enforce arc consistency using AC-3.
        Modifies self.domains in place.
        Returns True if consistent, False if any domain becomes empty.
        """
        # TODO: implement — use collections.deque for the queue
```

Integration: In your solve() method, call ac3() before backtracking begins. Print the reduced domains after AC-3 runs.

Expected AC-3 Output

```
AC-3 complete.
Reduced domains:
  WA  : ['R', 'G', 'B']
  NT  : ['R', 'G', 'B']
  SA  : ['R', 'G', 'B']
  ...
Domains unchanged (AC-3 did not prune any values for this problem)
```

Marks Breakdown — Task 3

13. revise() correctly removes unsupported values: 5 marks
14. ac3() uses a proper queue and re-enqueues affected arcs: 5 marks
15. Returns False on empty domain (detects unsolvable CSP): 3 marks
16. Printed domain output after AC-3 is correct: 2 marks

D — MRV and Degree Heuristics [10 Marks]

Improve variable selection in backtracking by implementing two heuristics inside **select_unassigned_variable()** when use_heuristics=True.

MRV — Minimum Remaining Values [6 Marks]

Select the unassigned variable with the **fewest remaining legal values** in its current domain (after any pruning by FC or AC-3). Rationale: fail early on the most constrained variable.

```
# Pseudocode
unassigned = [v for v in self.variables if v not in assignment]
return min(unassigned, key=lambda v: len(self.domains[v]))
```

Note: *When use_fc=True, the domains passed to MRV must reflect FC-pruned domains, not original domains.*

Degree Heuristic — Tie-Breaking [4 Marks]

When two or more variables have the same MRV score, break ties by choosing the variable involved in the **most constraints with unassigned variables** (highest degree). Rationale: prioritize the variable most likely to cause future failures.

```
# Pseudocode for tie-breaking
def degree(var, assignment):
    return sum(1 for (v1,v2) in self.constraints
               if var in (v1,v2)
               and (v1 not in assignment)
               and (v2 not in assignment))
```

Comparison Requirement

In your main() test block, for EACH test case run all three modes and print a comparison table:

```
Problem: 4-Queens
┌──────────────────────────────────────────┬──────────────┬──────────┐
│ Mode                                     │ Backtracks   │  Time    │
├──────────────────────────────────────────┼──────────────┼──────────┤
│ Backtracking only                        │      ?       │  ?ms     │
│ Backtracking + Forward Checking          │      ?       │  ?ms     │
│ Backtracking + FC + MRV + Degree         │      ?       │  ?ms     │
└──────────────────────────────────────────┴──────────────┴──────────┘
```

Marks Breakdown — Task 4

17. MRV correctly selects variable with minimum domain size: 4 marks
18. Degree heuristic correctly counts unassigned neighbors for tie-breaking: 3 marks
19. Comparison table printed for all test cases: 3 marks